课程设计计划：高级 AI 编程代理 — 机制、策略与实现

Context

用户是工程与算法双修的技术专家，其领导要求准备关于 Claude Code、OpenCode 及其底层模型的高级课程。通过前期讨论，我们已经提炼出核心框架：

1. **核心循环**：Intent → Grounding（感知现状）→ Planning → Action → Grounding（观察结果）→ Feedback
2. **每个环节都有策略空间**，抓住机制+关键策略就抓住全部
3. **AlphaGo/MCTS 类比**可作为理论升华的锚点
4. 课程需要同时体现**工程深度**（怎么实现的）和**算法深度**（为什么这么实现）

课程总纲

课程名称：掀起 AI 编程的”引擎盖” — 机制、策略与工程实现

课程定位

- **受众**：有编程经验的工程师、算法工程师、技术管理者
- **前置要求**：有 LLM 使用经验，了解基本的 prompt engineering
- **目标**：学员能理解 AI 编程代理的完整机制，能评估/选择/定制工具，能基于原理构建自己的方案

课程结构：4 大篇章 + 12 个模块

第一篇：原理篇 — 引擎是怎么转的（模块 0-2）

模块 0：核心机制 — AI 编程代理的”发动机”

工程视角：一个 AI 编程代理到底在做什么？拆解 Claude Code 处理一个真实任务的完整 trace。

算法视角：序贯决策框架。

主题	内容
0.1 从一个真实任务说起	录屏/trace 回放：给 Claude Code 一个 bug fix 任务，逐帧拆解它做了什么
0.2 核心循环	Intent → Grounding → Planning → Action → Grounding → Feedback 的完整推导
0.3 MDP 建模	将编程代理建模为马尔可夫决策过程 (S, A, T, R)，状态=代码库快照，动作=工具调用，奖励=测试结果
0.4 策略空间总览	每个环节的策略菜单（一张全景表），后续模块逐一展开
0.5 与 AlphaGo 的结构类比	策略网络≈LLM，价值网络≈测试/评估，MCTS≈搜索策略；类比成立的部分与裂缝

关键论文：ReAct (Yao 2022), Chain-of-Thought (Wei 2022)

实操：学员动手 trace 一个 Claude Code session，标注每一步属于循环的哪个环节

模块 1：Grounding — 感知的策略空间

核心问题：怎么让模型”看到”代码库？看什么、看多少、怎么看？

主题	内容
1.1 为什么 Grounding 是瓶颈	SWE-agent 的核心洞察：Agent-Computer Interface 比模型更重要

主题	内容
1.2 策略 1：暴力检索	Glob/Grep/全文件读取 — Claude Code 的方案，简单有效但消耗 token
1.3 策略 2：结构化感知	AST 解析、LSP 语义信息 — OpenCode 的方案，精准但实现复杂
1.4 策略 3：向量检索（RAG）	Embedding + 向量数据库，适合超大代码库，但有语义漂移风险
1.5 策略 4：混合方案	先粗检索再精读，分层感知策略
1.6 上下文窗口管理	有限窗口下的信息压缩、裁剪、优先级排序

关键论文：SWE-agent (Yang 2024), Agentless (Xia 2024)

工程实操： - 对比 Claude Code (Grep+Read) 与 OpenCode (LSP) 在同一任务上的 grounding 差异 - 测量不同策略的 token 消耗和信息覆盖率

模块 2：底层模型 — 引擎的”燃料”

核心问题：模型的哪些能力在支撑代理行为？

主题	内容
2.1 代码预训练	从 Codex 到 Claude：在代码上预训练为什么有效
2.2 指令遵循与 RLHF	模型怎么学会”听话”的 — RLHF 与 Constitutional AI
2.3 Tool Use 能力	函数调用的训练机制：从自然语言到结构化 JSON 输出
2.4 长上下文推理	128K/200K 窗口的技术实现与实际表现衰减
2.5 模型选择策略	Opus vs Sonnet vs Haiku：能力/成本/延迟三角；OpenCode 的多模型切换
2.6 基准测试的真相	SWE-bench、HumanEval 在测什么、没测什么、怎么被刷榜

关键论文：Codex (Chen 2021), RLHF (Ouyang 2022), Constitutional AI (Bai 2022), Toolformer (Schick 2023)

实操：同一个编程任务分别用 Opus、Sonnet、Haiku 完成，对比行为差异和成本

第二篇：策略篇 — 每个环节怎么做（模块 3-6）

模块 3：Planning — 规划的策略空间

核心问题：知道现状和目标后，怎么拆解路径？

主题	内容
3.1 扁平任务列表	最简方案：直接列出 TODO，线性执行
3.2 树状分解	Tree of Thoughts：展开多个方案分支，评估后选最优
3.3 DAG 依赖图	任务间有依赖关系时的拓扑排序执行
3.4 多代理分工	MetaGPT 式角色分工 vs AutoGen 式对话协作
3.5 Plan 的粒度	什么时候细规划、什么时候粗规划？过度规划的代价
3.6 动态重规划	执行过程中发现计划不对，怎么调整？

关键论文：Tree of Thoughts (Yao 2023), MetaGPT (Hong 2023), AutoGen (Wu 2023)

实操：Claude Code 的 Plan Mode 实战 — 让代理先规划、审批后执行

模块 4：Action — 执行的策略空间

核心问题：决定了做什么之后，怎么做？

主题	内容
4.1 策略 1：Tool Use (JSON 函数调用)	Claude Code 的核心方案：Read/Edit/Write/Bash 工具集的设计哲学
4.2 策略 2：CodeAct (代码执行)	用 Python/Shell 代码作为动作语言，比 JSON 更灵活
4.3 策略 3：直接生成	

主题	内容
	不调用工具，直接输出完整代码块，简单但缺乏验证
4.4 工具设计原则	什么样的工具接口对模型友好？从 SWE-agent 的 ACI 设计学到的
4.5 MCP 协议	Claude Code 的 MCP Server 扩展机制：怎么给代理”装新工具”
4.6 权限与沙箱	行动的安全边界：Claude Code 的权限模型 vs OpenCode 的设计

关键论文：CodeAct (Wang 2024), SWE-agent (Yang 2024)

实操： - 构建一个自定义 MCP Server（如数据库查询工具）并接入 Claude Code - 对比 Tool Use 和 CodeAct 两种策略在同一任务上的表现

模块 5：Feedback — 评估与修正的策略空间

核心问题：怎么知道做对了？做错了怎么办？

主题	内容
5.1 策略 1：外部验证	测试执行、Lint 检查、类型检查 — 最可靠但覆盖面有限
5.2 策略 2：模型自评	LLM 审查自己的输出 — Self-Refine 的机制
5.3 策略 3：Reflexion	从失败中学习：将错误经验转化为语言记忆，下次避免
5.4 策略 4：人在回路	Human-in-the-loop：什么时候该问人？Claude Code 的权限确认设计
5.5 Hooks 系统	Claude Code 的 pre/post hooks：在工具调用前后插入自动检查
5.6 失败恢复策略	重试 vs 回溯 vs 重新规划 vs 上报人类 — 怎么选？

关键论文：Self-Refine (Madaan 2023), Reflexion (Shinn 2023)

实操：配置 Claude Code 的 Hooks，实现”每次编辑后自动运行测试”

模块 6：搜索策略 — 从贪心到 MCTS

核心问题：当前编程代理的搜索策略处于什么水平？未来方向在哪？

主题	内容
6.1 搜索策略进化光谱	贪心（ReAct）→ 线性回溯（Reflexion）→ 树搜索（ToT）→ MCTS
6.2 当前主流为什么是贪心	实际工程约束：token 成本、延迟、状态管理复杂度
6.3 AlphaGo 深入解剖	MCTS 四步骤 (Selection/Expansion/Simulation/Backpropagation) 在代码场景的映射
6.4 代码空间搜索的特殊困难	无界状态空间、模糊评价函数、高分支因子
6.5 前沿进展	SWE-Search、CodeTree、LATS — 向系统化搜索迈进
6.6 “编程代理的 AlphaGo 时刻”	什么条件成熟了这一刻才会到来？

关键论文：LATS (Zhou 2023), SWE-Search (Antonis 2024), CodeTree (Li 2024)

实操：模拟实验 — 同一个 bug fix 任务，分别用贪心策略和树搜索策略，对比路径和效率

第三篇：工具篇 — 深入两大引擎（模块 7-8）

模块 7：高阶 Claude Code

核心问题：如何充分释放 Claude Code 的工程能力？

主题	内容
7.1 配置深度解析	CLAUDE.md 的设计哲学、settings.json 全字段详解、项目级 vs 全局配置的优先级
7.2 钩子（Hooks）系统	

主题	内容
	工具执行前/后置钩子的完整机制：用于 CI、Linting、安全网关
7.3 MCP 服务器	模型上下文协议详解：架构、通信机制、自定义工具开发（数据库、API、内部服务）
7.4 智能体团队（Agent Teams）	多智能体协作全流程：团队创建、任务分配与依赖、并行工作流、进度追踪、Shift+Down 切换
7.5 权限模型与安全	沙箱隔离机制、白名单配置、自主模式 vs 监督模式的选择策略
7.6 高级技巧	Plan Mode 工作流、/compact 上下文压缩、Session 恢复与 /resume、自定义 slash commands

实操： - 构建一个自定义 MCP Server 并接入 Claude Code - 配置 Hooks 实现 CI 自动门禁 - 用 Agent Teams 完成一个多文件功能开发

模块 8：高阶 OpenCode

核心问题：如何利用 OpenCode 的开放生态构建灵活的编程工作流？

主题	内容
8.1 架构与设计哲学	基于 Go 的 TUI 架构、隐私优先设计（不存储代码/上下文数据）、与 Claude Code 的设计差异
8.2 LSP 集成深度解析	语言服务器协议的工作原理、OpenCode 如何自动配置 LSP 为 LLM 提供结构化上下文
8.3 多模型提供商配置	Claude、OpenAI、Gemini、本地模型（Ollama）的接入与切换、API Key 管理
8.4 OpenCode Zen	精选模型组合的设计理念、针对编程任务的基准测试与模型推荐
8.5 多会话与协作	同一项目并行运行多个智能体、会话共享（通过链接协作调试）、Session 管理

主题	内容
8.6 扩展与二次开发	OpenCode 开源代码库结构、Plugin 机制、如何贡献或 Fork 定制

实操： - 安装并配置 OpenCode，接入至少 3 个不同模型提供商 - 用 LSP 集成完成一个跨文件重构任务，对比无 LSP 时的表现 - 使用会话共享功能进行协作调试

第四篇：实战篇 — 融会贯通（模块 9-11）

模块 9：编程智能体的提示词工程

核心问题：怎么跟编程代理”说话”才能让它高效工作？

主题	内容
9.1 系统提示词与指令文件	CLAUDE.md vs .opencode 配置对比、指令的层级与覆盖规则
9.2 高效的任务拆解	将复杂功能分解为对智能体友好的步骤、什么粒度最有效
9.3 上下文管理	该包含什么、上下文窗口溢出时的处理、上下文压缩策略
9.4 常见失效模式	API 幻觉、过度工程、陷入死循环、Token 浪费模式的识别与修正
9.5 高阶技巧	多轮对话中的引导策略、约束生成（“只改这个文件”）、让代理自我检查

实操：编写能够稳定生成”跨多个文件进行正确修改”的提示词，对比不同提示策略的成功率

模块 10：真实世界工作流与工程落地

核心问题：如何在真实团队和项目中落地 AI 编程代理？

主题	内容
10.1 独立开发者工作流	修复 Bug → 测试 → 提交代码 → 发起 PR，全流程由智能体驱动
10.2 团队集成	CI/CD 钩子、让智能体参与 Code Review、PR 自动化、与 GitHub/GitLab 的集成
10.3 Monorepo 与大型代码库策略	应对超大上下文、针对性的代码探索、分层 CLAUDE.md 配置
10.4 何时”不”该用智能体	了解局限性、典型失败场景、知道什么时候必须手动写代码
10.5 安全、隐私与企业级考量	数据流向（哪些上云、哪些留本地）、OpenCode 隐私优先 vs Claude Code 的方案、API 密钥管理、审计日志、合规性
10.6 成本控制	Token 用量监控、模型降级策略（Opus→Sonnet→Haiku）、缓存与去重

实操： - 使用两种工具完成一个端到端（End-to-End）的功能落地 - 为一个真实项目配置 CI/CD 集成的智能体工作流

模块 11：构建你自己的编程代理

核心问题：理解了机制和策略之后，怎么自己造一个？

主题	内容
11.1 从 API 开始	Claude API / Anthropic SDK 基础：tool_use、streaming、多轮对话
11.2 最小代理实现	100 行代码实现一个能读文件、编辑文件、跑测试的编程代理
11.3 加入 Grounding	给你的代理加上代码检索能力（Grep / AST / RAG 任选）
11.4 加入 Feedback	给你的代理加上测试驱动的自我修正循环
11.5 加入 Planning	给你的代理加上 Plan-then-Act 机制
11.6 作为平台的 OpenCode	

主题	内容
	扩展 OpenCode 开源代码库、自定义智能体、为团队设计特定领域的编程代理

实操（结业项目）：从零构建一个领域特定的编程代理，整合课程所有概念

附录：课程论文清单

基石层

论文	年份	核心贡献
Chain-of-Thought Prompting	2022	逐步推理，代理”思考”的基础
ReAct	2022	推理+行动交织，Agent 循环的起源
Toolformer	2023	LLM 自主使用工具的理论基础

反思层

论文	年份	核心贡献
Self-Refine	2023	生成→自评→改进的迭代循环
Reflexion	2023	语言记忆式经验学习
Tree of Thoughts	2023	推理从链式扩展为树状搜索

编程代理层

论文	年份	核心贡献
SWE-agent	2024	ACI 设计比模型更重要
CodeAct	2024	代码执行作为行动方式
Agentless	2024	简单两步流程的有效性

搜索策略层

论文	年份	核心贡献
LATS	2023	LLM 同时充当策略网络和价值网络
SWE-Search	2024	MCTS 在软件工程任务中的应用
CodeTree	2024	代码生成中的树搜索

多代理协作层

论文	年份	核心贡献
MetaGPT	2023	角色分工式多代理
AutoGen	2023	对话式多代理框架
ChatDev	2023	软件公司模拟

模型底座层

论文	年份	核心贡献
Codex	2021	代码预训练
RLHF (InstructGPT)	2022	人类反馈对齐
Constitutional AI	2022	原则约束的安全机制

课程形式建议

- **4 大篇章，12 个模块**（模块 0-11）
- 每模块 2-2.5 小时（共约 24-30 小时）
- 每模块结构：机制讲解（30%）+ 论文/算法解读（20%）+ 工程演示（20%）+ 动手实操（30%）
- 结业项目：从零构建一个领域特定的编程代理

核心一张图（课程开篇使用）

```
┌──────────────────────────┐
│     Intent (目⍰)          │
```

```
        └──────────────┬──────────────┘
                       ▼
        ┌──────────────────────────┐
┌──────►│  Grounding（感知□状）     │◄──┐
│       │  [Grep/AST/LSP/RAG]      │   │
│       └──────────────┬──────────────┘   │
│                      ▼                  │
│       ┌──────────────────────────┐      │
│       │  Planning（□划路径）      │      │
│       │  [Flat/Tree/DAG/Multi]   │      │
│       └──────────────┬──────────────┘   │
│                      ▼                  │
│       ┌──────────────────────────┐      │
│       │  Action（□行□更）        │      │
│       │  [ToolUse/CodeAct/Gen]   │      │
│       └──────────────┬──────────────┘   │
│                      ▼                  │
│       ┌──────────────────────────┐      │
│       │  Grounding（□察□果）      │      │
│       └──────────────┬──────────────┘   │
│                      ▼                  │
│       ┌──────────────────────────┐      │
└───────│  Feedback（□估/修正）     │──────┘
        │  [Test/Self/Reflect/Human]│
        └──────────────────────────┘
```